

Surprise Test

Exercise 1:

A Mersenne number is any number that can be written as $2^p - 1$ for some p . For example, 3 is a Mersenne number ($2^2 - 1$) as is 31 ($2^5 - 1$).

Write a function that accepts an exponent p and returns the corresponding Mersenne number.

```
# Write function for Mersenne number
def mersenne_number(p):
```

Mersenne numbers can only be prime if their exponent, p , is prime. Make a list of the Mersenne numbers for all primes p between 3 and 65 (there should be 17 of them).

```
# To check if a number is prime
def is_prime(number):
```

```
# To get the list of prime numbers between two given number (n_start < n_end)
def get_primes(n_start, n_end):
```

Now, make use of the functions you've defined above to create the appropriate list of Mersenne numbers.

```
mersennes = []
```

Exercise 2:

Exercise 2: Optimize `is\_prime`

You might have noticed that the primality check `is\_prime` we developed before is somewhat slow for large numbers. This is because we are doing a ton of extra work checking every possible factor of the tested number. We will use two optimizations to make a `is\_prime\_fast` function.

The first optimization takes advantage of the fact that two is the only even prime. Thus we can check if a number is even and as long as it's greater than 2, we know that it is not prime.

Our second optimization takes advantage of the fact that when checking factors, we only need to check odd factors up to the square root of a number. Consider a number n decomposed into factors $n=ab$. There are two cases, either n is prime and without loss of generality, $a=n$, $b=1$ or n is not prime and $a, b \neq n, 1$. In this case, if $a > \sqrt{n}$, then $b < \sqrt{n}$. So we only need to check all possible values of b and we get the values of a for free! This means that even the simple method of checking factors will increase in complexity as a square root compared to the size of the number instead of linearly.

Lets write the function to do this and check the speed! ``is_prime_fast`` will take a number and return whether or not it is prime.

You will see the functions followed by a cell with an ``assert`` statement. These cells should run and produce no output, if they produce an error, then your function needs to be modified. Do not modify the assert statements, they are exactly as they should be!

Run the following to make sure it finds the same primes as the original function.

```
for n in range(10000):  
    assert is_prime(n) == is_prime_fast(n)
```

Now lets check the timing, here we will use the `%%timeit` magic which will time the execution of a particular cell.

```
%%timeit  
is_prime(67867967)
```

```
%%timeit  
is_prime_fast(67867967)
```

Now, write a function which will find all prime numbers up to and including n .

```
def get_primes_fast(n):  
    return [# write your code here]
```